DAY 01 STUDY NOTES • PHASE 0

Day 01 – HTML Fundamentals

Comprehensive Core Learning Notes, System Architecture & Practical Revision

Student: Shreyas Kumbhar
Focus: Java/Spring Boot to React + Next.js
Mode: Deep Conceptual Mastery

Executive Study Blueprint & Objective

This document serves as the permanent revision manual for **Day 01** of your 123-day engineering transition. Rather than rote syntax memorization, this guide grounds HTML within full-stack engineering paradigms, mapping frontend interfaces directly to your Spring Boot REST backend mental models, accessibility primitives, and DOM hierarchy trees.

Master Table of Contents

Foundational Architecture & Syntax

- **Topic 1:** Frontend & Backend Integration (Spring Boot Connection)
- **Topic 2:** What is HTML? Tags, Elements & Void Tags
- **Topic 3:** Basic HTML Document Structure & Shell
- **Topic 4:** HTML Elements, Attributes & Data Passing
- **Topic 5:** Semantic HTML & Layout Architecture
- **Topic 6:** Text Structure: Headings Hierarchy & Paragraphs
- **Topic 7:** HTML Lists (Ordered, Unordered & Data Grouping)

Forms, DOM, Project & Review

- **Topic 8:** Hyperlinks & Image Semantics (Relative vs Absolute)
- **Topic 9:** HTML Forms & Input Types Master Reference
- **Topic 10:** Accessible Form Coupling (`for` ↔ `id`)
- **Topic 11:** Multi-Line Data Entry with Textarea
- **Topic 12:** Form Action, HTTP Methods & Backend API Flow
- **Topic 13:** Practical Web Accessibility Fundamentals (a11y)
- **Topic 14:** The Browser DOM Tree & Node Hierarchies

Engineering Milestone: Phase 0 Foundations

Total Phase 0 Days	Daily Code Split	Day 01 Target
10 Days	**30% Read / 70% Code**	**Semantic Resume**
HTML & CSS Foundations	Active Typing Practice	Pure HTML + Form + Accessibility

Guiding Rule for Java Developers: Think of HTML not as visual styling, but as *structured data modeling* for the browser. A properly marked-up HTML document is analogous to a well-typed Java class schema: every element describes the semantic nature of the data it holds before any presentation layer (CSS) or reactive behavior (JS/React) is applied.

Topic 1: Introduction to Frontend and Backend

In web software engineering, applications are separated into presentation concerns (Client/Frontend) and data persistence/business rules (Server/Backend). As a Java Spring Boot engineer, your mental model operates around controllers, service layers, entity persistence, and JSON payloads. Transitioning to full-stack requires understanding how the client environment renders and interacts with those services.

Full-Stack Request-Response Flow (Conceptual Client-Server Cycle)

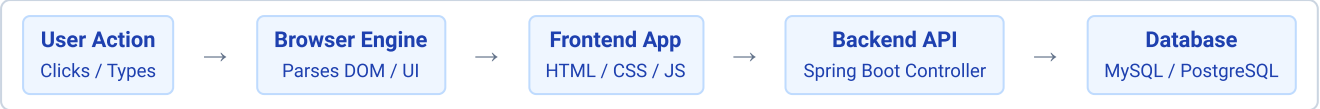


Figure 1.1: Simplified representation of client-server interaction across the web stack.

The Core Web Trio vs. Backend Engineering

Layer	Core Responsibility	Analogy in Physical Construction	Spring Boot Parallel
HTML	**Structure:** Defines elements, hierarchy, text, headings, and data entry forms.	The wooden framing, concrete foundation, and bare walls of a house.	The entity data model or DTO schema defining fields and data contracts.
CSS	**Styling:** Dictates colors, spacing, layout alignments, fonts, and responsiveness.	The interior paint, wallpaper, lighting fixtures, and furniture styling.	Response formatting, view serialization, or UI display decorators.
JavaScript	**Behavior:** Dynamic interactivity, client validation, asynchronous fetches.	The electrical wiring, switches, plumbing valves, and home automation.	Client-side business logic, event handlers, and data transformation routines.
Backend	**Business Logic:** Authentication, transactional security, database storage.	The city power grid, municipal water treatment, and central banking system.	Spring Boot Services (`@Service`), Spring Data Repositories, DB.

Topic 2: What Is HTML?

HTML stands for **HyperText Markup Language**. It is not a procedural or object-oriented programming language; rather, it is a declarative markup language that annotates text and assets so that browser rendering engines can structure them into a meaningful visual document.

HTML Tag vs. HTML Element

An **HTML Tag** is the syntactic bracket marker (`<p>` or `</p>`).
An **HTML Element** is the complete construct, consisting of the opening tag, any attributes, the enclosed inner content, and the closing tag.

Void / Self-Closing Elements

Elements that cannot contain child nodes or inner text are called **void elements**. They have no closing tag. Examples include `<img>` , `<input>` , `<br>` , `<hr>` , and `<meta>` .

Deconstruction of Core Syntax Examples

```
      <!-- Standard Element: Opening Tag + Content + Closing Tag --> <p>Hello World</p> ^ ^ ^ |
| +-- Closing Tag (indicates end of paragraph node) | +-------------- Content (raw text payload d
isplayed to user) +----------------- Opening Tag (declares node type 'p') <!-- Void / Self-Closi
ng Element: Tag Name + Attributes + Attribute Values --> <img src="profile.jpg" alt="Profile phot
o"> ^ ^ ^ ^ ^ | | | | +-- Attribute Value (alternative text for screen readers/fallback) | | | +-
----- Attribute Name (accessibility description) | | +----------------- Attribute Value (URL or
local path to image source) | +---------------------- Attribute Name (source locator) +----------
---------------- Void Tag Name (contains no closing </img> or inner text)
```

Topic 3: Basic HTML Document Structure

Every HTML document requires an explicit outer skeleton. Without this baseline hierarchy, web browsers enter non-standard "quirks mode," causing unpredictable layout calculation and rendering anomalies.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>My Resume</title> </head> <body> <header> <h1>My Resume</h1> </header> <main> <section> <h2>About Me</h2> <p>I am learning frontend development.</p> </section> </main> <footer> <p>© 2026 My Resume</p> </footer> </body> </html>
```

Critical Structural Node Roles

- `<!DOCTYPE html>`: Tells browser to render under modern HTML5 standards.
- `<html lang="en">`: Root wrapper; attributes define document human language.
- `<head>`: Metadata container (character set, viewport, page title, links).
- `<body>`: All visual content displayed to the user inside the browser viewport.

Vital Semantic Distinctions

- **`<head>` vs `<header>`**: `<head>` contains invisible page metadata. `<header>` is a visible landmark for top banners/titles.
- **`<body>` vs `<main>`**: `<body>` encloses all visible elements. `<main>` marks the single unique primary content area.

Topic 4: HTML Elements and Attributes

Attributes provide configuration, behavioral switches, or identity metadata to elements. Attributes are **always declared inside the opening tag** and are formatted as `name="value"` key-value pairs.

Attribute	Applies To	Purpose & Functionality	Code Example
`href`	`<a>`, `<link>`	Hypertext Reference: defines destination URL or internal page fragment.	`<a href="https://example.com">`
`src`	`<img>`, `<script>`	Source path: specifies file location or URL of media to embed.	`<img src="profile.jpg" alt="...">`
`alt`	`<img>`	Alternative text for screen readers and when images fail to load.	`<img src="pic.png" alt="User avatar">`
`id`	Global (all elements)	Unique document identifier. Used for CSS styling, JS access, and anchors.	`<section id="experience">`
`name`	Form fields (`input`, etc.)	Key sent in HTTP payload upon form submission (`key=value`).	`<input name="userEmail">`
`type`	`<input>`, `<button>`	Determines UI input control behavior (e.g. text, email, submit).	`<input type="email">`
`for`	`<label>`	Associates label with a target input element's unique `id`.	`<label for="userEmail">`

Topic 5: Semantic HTML & Document Architecture

Semantic HTML refers to using elements that convey their computational meaning to the browser, assistive technologies (screen readers), and search crawlers, rather than using generic layout dividers. Non-semantic elements like `<div>` (block container) and `<span>` (inline container) carry zero informational meaning.

Simplified Semantic Hierarchy Tree

```
html ├── head (Metadata) └── body (Visible UI) ├── header (Intro / Masthead) ├── nav (Prima
```

Note: Simplified conceptual guide; not every page requires all elements.

Topic 6: HTML Headings and Paragraphs

Headings (`<h1>` through `<h6>`) establish an indexed outline hierarchy for the page. Screen readers allow visually impaired users to jump directly across sections using heading levels, much like navigating a technical book index.

```
<h1>My Resume</h1> <!-- Top-level document subject (One per page) --> <section> <h2>Educati
on</h2> <!-- Major section boundary --> <h3>Computer Science Degree</h3> <!-- Sub-topic nested un
der Education --> <p>Studied data structures, algorithms, and backend systems.</p> </section>
```

Rule 1: Never Select Headings for Visual Size

Do not choose `<h4>` because you want smaller text or `<h1>` because you want big text. Visual dimensions belong

Rule 2: Avoid Skipping Heading Levels

Heading levels should follow a structured sequence (`<h1>` → `<h2>` → `<h3>`). Skipping from `<h1>` straight to

Topic 7: HTML Lists

Lists group related items into structured collections. They maintain parent-child relationships where the parent list (`<ul>` or `<ol>`) directly wraps list item children (`<li>`).

Unordered Lists (`<ul>`) - Order Doesn't Matter

```
<ul> <li>Java</li> <li>Spring Boot</li> <li>HTML</li> </ul>
```

Used for technical skillsets, feature highlights, and navigation links.

Ordered Lists (`<ol>`) - Sequential Steps

```
<ol> <li>Learn HTML Fundamentals</li> <li>Learn CSS Layouts</li> <li>Master JavaScript ES6+</li> </ol>
```

Used for chronological workflows, instructions, and ranked items.

Topic 8: Hyperlinks and Images

Hyperlinks (`<a>`) and images (`<img>`) connect documents and embed graphical assets:

Links: `<a href="...">`

Absolute URLs: Point to external web domains (e.g., `<a href="https://github.com">GitHub</a>`).
Relative URLs: Point to internal files (e.g., `<a href="/about.html">About</a>`) or internal fragment IDs (`<a href="#contact">Contact</a>`).

Images: `<img src="..." alt="...">`

The `src` attribute specifies image location. The `alt` attribute provides alternative text. *Crucial clarity:* `alt` is an image-only attribute; it is not a general description field for all HTML elements.

Topic 9: HTML Forms Overview & Input Controls

An HTML Form (`<form>`) is the primary interactive container through which users enter data to be sent to a server. Form inputs translate user input into key-value pairs for HTTP transport.

Input Type	Purpose & Behavior	Browser Validation	Code Syntax
`text`	Single-line general text input.	None (accepts any string).	`<input type="text">`
`email`	Single-line email address input.	Enforces valid format (`@` and domain).	`<input type="email">`
`password`	Masks characters for credential entry.	None (visual obfuscation only).	`<input type="password">`
`number`	Numerical input with step spinners.	Restricts input to numbers.	`<input type="number">`
`checkbox`	Binary on/off toggle (can pick multiple).	Checks if checked if marked required.	`<input type="checkbox">`
`radio`	Single selection within a shared `name` group.	Allows selecting only one in group.	`<input type="radio">`
`submit`	Renders button that submits enclosing form.	Triggers native form validation.	`<button type="submit">`

Topic 10: Label and Input Relationship

Form accessibility requires explicitly binding each `<label>` to its respective `<input>`. This coupling ensures screen readers announce the input's purpose when focused and allows users to click the label text to focus the field.

```
<label for="email">Email Address:</label> <input type="email" id="email" name="email" requi
red placeholder="Enter your email"> | | | | +--- Matches ------+ | | +-- Temporary visual hint
(Couples Label to Input) | +-- Native browser validation flag +-- Key sent in HTTP payload (e.g.
email=john%40test.com)
```

`id` vs. `name` : The Crucial Difference

`id` (Client-side identifier): Unique to the DOM document. Used by `<label for="...">`, CSS, and JavaScript.

`name` (Server-side parameter key): The parameter key included in the HTTP request payload when the form is submitted.

`required` vs. `placeholder`

`required` : A boolean validation attribute that blocks form submission if empty.

`placeholder` : Transient sample text inside the field. *Never use placeholder as a substitute for a label*; it vanishes when the user types!

Topic 11: Textarea (Multi-Line Text Input)

Unlike the self-closing `<input>`, the `<textarea>` element is designed for multi-line text input (such as comments or messages) and requires a closing tag.

```
<label for="message">Message:</label> <textarea id="message" name="message" rows="4" maxlen
gth="200" placeholder="Enter message" required></textarea>
```

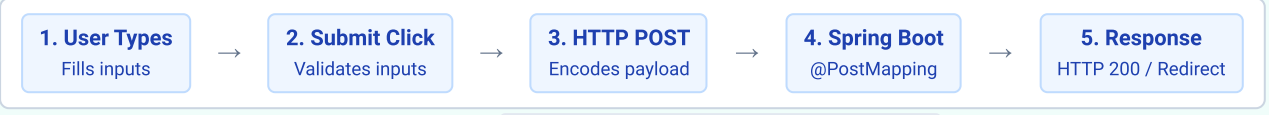
Topic 12: Form Action, Method & Spring Boot Integration

The `<form>` element specifies where and how user data travels across the network:

`action="/contact"` : The target URL or API endpoint handling the request.

`method="POST"` : The HTTP verb (GET appends data to URL query parameters; POST puts data in request body).

Full Lifecycle: Browser Form Submission → Spring Boot Backend



Spring Boot Connection: When you submit a form with `action="/contact" method="POST"`, the browser sends an `application/x-www-form-urlencoded` HTTP POST request to your Spring Boot controller matching `@PostMapping("/contact")`. The form's `name` attributes map directly to method parameters annotated with `@RequestParam` or fields in a Java form DTO object.

Topic 13: Accessibility Fundamentals (a11y)

Web Accessibility ensures that websites can be navigated, operated, and understood by everyone, including people using screen readers, keyboard-only navigation, and other assistive tools. Accessibility is not an afterthought; it starts directly with writing clean, semantic HTML markup.

Accessibility Principle	Incorrect Pattern (Inaccessible)	Correct Pattern (Accessible)
Form Input Labels	`<input placeholder="Enter Name">` Screen reader cannot announce name reliably; placeholder vanishes.	`<label for="uname">Name:</label> <input id="uname" ...>` Explicit association announced to screen reader; clicking label focuses input.
Image Alternatives	`<img src="chart.png">` Screen reader announces raw file URL or skips image entirely.	`<img src="chart.png" alt="Quarterly Revenue Growth">` Provides concise, descriptive text alternative of the graphic.
Interactive Buttons	`<div onclick="submit()">Submit</div>` Cannot be reached via keyboard Tab key; missing button role.	`<button type="submit">Submit</button>` Keyboard focusable, triggered with Enter/Space, native semantics.
Document Landmarks	`<div class="header"><div class="main">` Screen reader cannot jump between page regions.	`<header> <main> <footer>` Exposes landmark regions that assistive software can jump between directly.

> **Important Engineering Distinction:** While Semantic HTML is the foundation of accessible web engineering, semantic markup alone does not make a site 100% accessible. Comprehensive accessibility also requires proper color contrast, keyboard tab ordering, ARIA live regions for dynamic updates, and screen reader testing.

Topic 14: DOM Introduction (Document Object Model)

When the browser downloads an HTML document, it reads the characters, parses the tags into tokens, and builds an in-memory tree representation called the **Document Object Model (DOM)**. Each HTML element becomes a node in this tree with explicit parent, child, and sibling relationships.

Raw HTML Code

```
<body><header><h1>My Resume</h1></header>
```

Parsed DOM Tree Hierarchy

```
body (Root visible container) ├── header (Child of body, sibling of main) │ └── h1 (Ch
```

Parent: An element wrapping another. Child: Nested inside. Sibling: Shares the same parent.

Chapter 3: Practical Project – Semantic HTML Resume

During Day 01, you practiced structuring a complete, valid semantic resume document. *Note: This is an educational study model demonstrating clean hierarchy, structured sections, and accessible forms. It is not tied to an active Spring Boot server.*

```
<!DOCTYPE html>
<html lang="en">
<head> <meta charset="UTF-8">
<meta name="viewport" content="width=device-w
```

Document DOM Tree Hierarchy

```
html ├── head │ ├── meta [charset] │ ├── meta [viewport] │ └── title └── body ├── header │
```

Section-by-Section Engineering Breakdown

- **Header (`<header>`):** Declares the primary page subject using a single, document-level `<h1>` .
- **Sections (`<section>`):** Divides independent thematic blocks (Skills, Education, Contact). Each maintains an `<h2>` to create a clear document outline.
- **List (`<ul>`):** Uses unordered list items (`<li>`) to model skills without implying an arbitrary numeric ranking.
- **Contact Form (`<form>`):** Every input and textarea is paired with a matching `<label for="...">` , uses `required` validation, and defines a `name` attribute for backend payload serialization.

Chapter 4: HTML Hierarchy as Parent-Child Sets

Hierarchical nesting is the core architecture of HTML. The following tree structures illustrate the relationships across the major concepts covered in Day 01:

1. Document Shell & Content Hierarchy

```
HTML ├── Document Structure │ ├── doctype (Standard mode) │ ├── html (Root wrapper) │ ├── head (Met
```

Chapter 5: Important Mistakes and Corrections

Common Pitfall	Incorrect Syntax	Correct Syntax & Engineering Rule
Confusing `<head>` & `<header>`	`<head><h1>My Resume</h1> </head>`	`<header><h1>My Resume</h1> </header>` `<head>` is strictly for invisible metadata; `<header>` is visible UI.
Confusing `<body>` & `<main>`	`<main>...</main>` without `<body>`	`<body><main>...</main></body>` `<body>` wraps all visible content; `<main>` sits inside body.
Missing `<label>` Binding	`<label>Email</label><input id="email">`	`<label for="email">Email</label> <input id="email">` Label's `for` must match input's `id` to enable screen-reader linking and click-to-focus.
Confusing `id` & `name`	`<input id="email">` (missing `name`)	`<input id="email" name="email">` Inputs without a `name` attribute are ignored during form submission.
Missing Image `alt`	`<img src="avatar.jpg">`	`<img src="avatar.jpg" alt="Profile avatar">` Always supply descriptive `alt` text for screen readers and broken image states.
Placeholder as Label Replacement	`<input placeholder="Enter Email">`	`<label for="em">Email:</label> <input id="em" placeholder="...">` Placeholders vanish on user typing and do not satisfy accessibility standards.
Assuming Action creates Backend	`<form action="/submit">` and expecting data to save automatically	Understand that `action="/submit"` only directs where the browser sends the HTTP request. A backend controller (`@PostMapping("/submit")`) must be implemented to process it.

Chapter 6: Day 01 Revision & Interview Questions

Review these 28 questions to consolidate your knowledge and prepare for frontend and full-stack technical interviews. Detailed answers follow on the next page.

Part 1: Basic HTML Architecture (Questions 1–10)

1. What is HTML and what is its specific responsibility in web engineering?
2. What is the precise structural difference between an HTML tag and an HTML element?
3. What is an HTML attribute and where is it declared in markup?
4. What is the explicit function of the `<!DOCTYPE html>` declaration?
5. What is the difference between the `<head>` and `<header>` elements?
6. What is the difference between the `<body>` and `<main>` elements?
7. Define Semantic HTML and explain why modern developers prioritize semantic tags.
8. What is the purpose of the `<section>` element, and how does it differ from a `<div>`?
9. What is the structural and semantic difference between `<ul>` and `<ol>`?
10. What is the purpose of the `<a>` element and what does its `href` attribute represent?

Part 2: HTML Forms & Data Transport (Questions 11–20)

11. What is an HTML form and what role does it serve in user interaction?
12. What is the purpose of the `<label>` element in form design?
13. What is the crucial functional difference between the `id` and `name` attributes on an input?
14. What behavior does the `required` attribute enforce during form submission?
15. What is the purpose of the `placeholder` attribute, and why must it never replace a label?
16. When would you use a `<textarea>` instead of an `<input type="text">`?
17. What is the difference between HTTP GET and POST methods in form submission?
18. What is the purpose of the `action` attribute on a `<form>` tag?
19. Why must the `<label>`'s `for` attribute strictly match the `<input>`'s `id` attribute?
20. How does a browser package and send form data upon user submission?

Part 3: Accessibility & DOM Fundamentals (Questions 21–28)

21. What is web accessibility (a11y) and why is it important in software development?
22. Why is the `alt` attribute mandatory for images, and how does assistive technology use it?
23. How do semantic elements assist screen-reader navigation across large websites?
24. What is the Document Object Model (DOM) conceptually?
25. What is a parent element in DOM hierarchy?
26. What is a child element in DOM hierarchy?
27. What is a sibling element in DOM hierarchy?
28. How does an HTML text file become a live rendered tree in memory?

Chapter 6 (Cont.): Revision Questions Answer Key

Answers: Basic HTML (1–10)

1. HTML: Standard markup language for structuring web pages; defines data containers, text, and inputs.
2. Tag vs. Element: A tag is the markup bracket (`<p>`); an element is the full unit including opening tag, content, and closing tag.
3. Attribute: Key-value parameter inside opening tags providing configuration or metadata (`name="val"`).
4. DOCTYPE: Tells the browser to render using modern HTML5 standards mode, preventing quirks mode.
5. Head vs. Header: `<head>` holds non-rendered metadata; `<header>` is a visible landmark for page or section introductions.
6. Body vs. Main: `<body>` wraps all visible content; `<main>` wraps only the primary, non-repeating page content.
7. Semantic HTML: Markup using tags that describe their meaning (e.g. `<article>`), improving accessibility, SEO, and developer readability.
8. Section: A generic thematic grouping of content, typically paired with a heading; unlike `<div>` , it has semantic meaning.
9. UL vs. OL: `<ul>` is for bulleted lists where item order is arbitrary; `<ol>` is for numbered sequential steps.
10. Anchor & href: `<a>` creates hyperlinks; `href` designates the target destination URL or anchor fragment.

Answers: Forms & Network (11–15)

11. Form: An interactive container collecting user inputs to submit to a server endpoint.
12. Label: Accessible textual label for an input; clicking it focuses the associated field.
13. ID vs. Name: `id` is a client-side DOM identifier; `name` is the parameter key sent in the HTTP payload to the server.
14. Required: A boolean validation attribute that blocks form submission if the field is empty.
15. Placeholder: Transient sample text inside an input. It disappears on focus/typing and does not replace a permanent accessible label.

Answers: Forms & Network (16–20)

16. Textarea: Used for multi-line text input (comments, messages); unlike `<input>` , it requires an explicit closing tag.
17. GET vs. POST: GET appends data to the URL query string (visible, length-limited, used for searches); POST sends data in the HTTP request body (used for data creation/mutations).
18. Action: The server-side URL endpoint where the browser sends the serialized form data.
19. For ↔ ID: Matches the label to the input node in the accessibility tree, allowing screen readers to announce the label when the field is focused.
20. Form Submission: The browser gathers inputs with `name` attributes, serializes them (typically as URL-encoded data), and sends an HTTP request using the specified method and action URL.

Answers: Accessibility & DOM (21–28)

21. Accessibility (a11y): Designing web apps so that people with disabilities can perceive, navigate, and interact with them effectively.
22. Alt Attribute: Provides alternative text for images; screen readers read it aloud, and browsers display it if images fail to load.
23. Semantic Landmarks: Assistive software uses landmark tags (`<main>` , `<nav>`) to let users jump directly across page regions.
24. DOM: An in-memory, tree-structured object representation of the HTML document created by the browser engine.
25. Parent Node: An element that directly encloses one or more other elements.
26. Child Node: An element directly enclosed within another element.
27. Sibling Node: Elements that share the exact same immediate parent element.
28. HTML to Tree: The browser parses raw HTML characters into tokens, creates DOM nodes, and links them into an hierarchical parent-child tree structure.

Chapter 7: Practical Exercises & Debugging

Exercises for Hands-on Practice

Ex 1 (Document Shell): Write a clean HTML5 template with `<!DOCTYPE html>`, utf-8 charset, viewport, and title.
Ex 2 (Semantic Structure): Build a 3-section layout using `<header>`, `<main>`, and `<footer>` with appropriate heading hierarchy.
Ex 3 (Contact Form): Create a form with Name, Email, and Message fields, properly bound labels (`for` ↔ `id`), and a submit button.
Ex 4 (DOM Diagram): Draw the DOM tree for a `<main>` containing two `<section>`s, each with an `<h2>` and a `<p>`.

Exercise 5: Debug HTML (Spot & Fix the Bugs)

1. `<img "profile.png" description="photo">`
→ *Fix:* `<img src="profile.png" alt="photo">`
2. `<label id="email">Email:</label><input name="email">`
→ *Fix:* `<label for="email">Email:</label><input id="email" name="email">`
3. `<a url="https://github.com">Git</a>`
→ *Fix:* `<a href="https://github.com">Git</a>`
4. `<p><h2>Title</h2></p>`
→ *Fix:* Block headings cannot be nested inside `<p>` elements.
5. `<textarea value="Type here">`
→ *Fix:* Textarea content sits between opening/closing tags, not in a `value` attribute.

Chapter 8: Day 01 Learning Summary & Self-Assessment Checklist

Day 01 Self-Assessment Checklist (Verify each item before moving forward)

- [ ] Understand frontend vs backend separation of concerns
- [ ] Understand the distinct roles of HTML, CSS, and JavaScript
- [ ] Understand tags vs elements vs attributes
- [ ] Understand void/self-closing elements (`img` , `input`)
- [ ] Can write the complete HTML5 document shell from memory
- [ ] Know the difference between `<head>` and `<header>`
- [ ] Know the difference between `<body>` and `<main>`
- [ ] Can use semantic elements (`header` , `main` , `section` , `footer`)
- [ ] Understand heading levels (`h1` through `h6`) and outline hierarchy
- [ ] Can build ordered (`ol`) and unordered (`ul`) lists
- [ ] Know absolute vs relative links with the `<a>` element
- [ ] Know why the `alt` attribute is essential for images
- [ ] Can build accessible forms using `<form>` and `<input>`
- [ ] Understand `<label for="...">` coupling with `<input id="...">`
- [ ] Know the difference between client `id` and server `name`
- [ ] Understand input types (`text` , `email` , `password` , etc.)
- [ ] Know when to use `<textarea>` for multi-line inputs
- [ ] Understand form `action` and `method` (GET vs POST)
- [ ] Understand how form data flows into a Spring Boot API
- [ ] Can visualize HTML elements as a DOM tree

Next Learning Step

Continue reinforcing your **HTML fundamentals and semantic document structure** through hands-on practice. Build variations of your semantic resume and contact forms until writing well-structured markup feels second nature. Once your HTML foundation is solid, you will move forward to CSS styling on Day 02 according to your roadmap.